Part - 2

String

1. Why is the String class immutable in Java and what practical advantages does this provide?

Ans:

The string class is immutable in java meaning that once a string object is created its value cannot be changed. This immutability provides several practial advantages.

i) Security: Prevents sensitive data from being modified.

ii) Thread Safety: Multiple threads can safely share the same string object.

iii) Memory Efficiency: Enables the use of the

string pool, reducing memory usage.

iv) Performance: Allows caching of hash codes, improving performance in collections like HashMap.

v) Reliability: Makes programs easier to maintain and debug.

2. What is the difference between creating a String using a literal versus using the new keyword, and how does the String pool affect this?

Ans:

String Literal	Using new keyword
Created using double quotes (" ")	Created using the "new" keyword
Stored in the String Pool	Stored in the Heap memory as a new object.

The String pool is a special memory area that stores unique string literals. When a string literal is created, Java first checks the string pool. If the string already exists, Java reuses the existing object instead of creating a new one which saves memory and improves performance.

3. Why does comparing Strings with == often given unexpected results, and why should .equals() be used instead?

Ans:

The == operator compares the memory addresses of two string objects, not their actual contents. Therefore two string with the same text may produce false if they are stored in different memory locations.

The .equals() method compares the actual contents of the string. so it should be used when checking whether two string contains the same characters.

== operator

i) Compares memory address.

ii) May give unexpected result.

.equals() method

i) Compares the actual contents of Strings.

ii) The correct result for content comparison.

4. What is the difference between String, StringBuilder and StringBuffer and when would you choose one over the others.

Ans:

String	StringBuilder	StringBuffer
Immutable	Mutable	Mutable
Thread-safe Yes	Thread-safe No	Thread-safe Yes
Slow for frequent modification	Fastest	Slower
Usage Fixed text	Frequent modifications in a single-threaded environment	Frequent modifications in a multi-threaded environment

1) String: Use when the text does not change frequently.

ii) StringBuilder: Use when performing many string modifications in a single-threaded program.

iii) StringBuffer: Use when performing many string modifications in a multi-threaded program and thread safety is required.

Scanner

1. What is the purpose of the scanner class and why is it commonly used for taking user input in java?

Ans:

The scanner class is used to read input from various sources, such as the

keyboard, files and strings. It is commonly used for taking user input because it provides simple and convenient methods to read different types of data, such as integers, floating-point numbers and strings.

Used:

i) It is easy to use.

ii) It supports multiple data types

iii) It provides built-in method for parsing input.

iv) It allows interactive input from the keyboard using System.in.

next()

i) Reads a single word.

ii) Reads characters until a space or newline is encountered.

nextLine()

i) Reads an entire line.

ii) all characters until the newlink character (\n)

nextInt()

i) Integer value

ii) Integer and leaves the newline character in the input buffer.

Issues:

1) nextInt() and next() do not consume the newline character (\n) entered after

the input.

ii) If nextLine() is called immediately after nextInt() or next(). It may read the leftover newline character instead of waiting for new input.

3. Why does a nextInt() followed by nextLine() sometimes appear to skip input and how would you explain this behavior?

Ans:

The nextInt() method reads only the integer value and leaves the newline character (\n) in the input buffer.

When nextLine() is called immediately afterward. It reads this leftover newline character as an empty line.

Array

1. What is an array in java and how does it differ from a collection like ArrayList in terms of size flexibility and type handling?

Ans:

An array in java is a data structure that stores multiple values of the same data type in a contiguous block of memory. Array have a fixed size, which means

their length cannot be changed after creation. In contrast an arraylist is a dynamic collection that can grow or shrink during program execution.

In terms of type handling, arrays can store primitive data types and object references. ArrayList however can only store objects and uses wrapper classes for primitive values through autoboxing. Additionally, arrays provide better performance due to direct memory access, while ArrayList offers greater flexibility and a richer set of built-in methods.

2. Why are arrays in java considered objects and what does this imply about default values and memory allocation?

Ans:

Array in java are considered objects because they are created dynamically and inherit properties and method from the Object class. This means that arrays are allocated memory on the heap and are accessed through references.

Because array are objects their elements are automatically initialized with default values when memory is allocated. Numeric

types are initialized to zero, boolean values to false and object references to null. The memory required for the entire array is allocated when the array object is created and the size of the array remains fixed throughout its lifetime.

3. How does passing an array to a method work in Java (pass-by-value vs pass-by-reference behavior) and what does this mean for modifying array contents inside a method?

Ans:

Java uses pass-by-value semantics for all method arguments including arrays. When an array is passed to a method the value being passed is a copy of the reference to the array object not a copy of the actual array.

As a result both the caller and the called method refer to the same array object in memory. Therefore modifications made to the contents of the array inside the method are visible outside the method. However reassigning the array reference

inside the method affects only the local copy of the reference and does not change the original reference in the calling code.

Package

1. What is a package in Java, and why is it important for organizing large code bases?

Ans:

A package in Java is a mechanism used to group related classes, interfaces and sub-packages together.

It provides a structured way to organize code and helps developers manage large applications more efficiently. Packages prevent naming conflicts by allowing classes with the same name to exist in different packages. They also improve code readability, maintainability and reusability by logically separating different components of a software project. In large codebases packages make it easier for developers to navigate manage and collaborate on projects.

2. What is the difference between built-in packages (like java.util) and user-defined packages?

Ans: Built-in packages are predefined package provided by java that contain ready-to-use classes and interfaces for various programming tasks. These packages are included in the java standard Library and offer functionalities such as data structure input/output operations, networking and utility fuctions. On the other hand, user-defined packages are created by programmers to

organize their own classes and interfaces according to the requirements of a specific application. While built-in packages are developed and maintained by Java, user-defined packages are designed and managed by developers.

3. How does the import statement work, and why is it not strictly required if fully qualified names are used instead?

Ans:

The import statement in Java allows

programmers to access classes and interfaces from other packages without writing their complete package paths every time. It simplifies the code and improves readability by enabling direct use of class names. However, the import statement is not strictly required because Java allows the use of fully qualified names, where the complete package and class name are specified whenever a class is referenced.

Using fully qualified names eliminates the need for import statements, although it can make the code longer and less readable.

4. What role do packages play in access control, particularly with default (package-private) access modifiers?

Ans:-

Package play an important role in java's access control mechanism. When a class member is declared without any explicit

access modifier, it receives default or package-private access. This means that the members can only be accessed by other classes within the same package and cannot be accessed from classes outside that package. This access level helps enforce encapsulation by restricting visiblity to related classes within a package, thereby improving security and maintaining proper seperation of components in a Java application.

Encapsulation

1. What is encapsulation in OOP, and how does it help in achieving data hiding?

Ans:

Encapsulation is one of the fundamental principles of Object Oriented Programming (OOP) that involves combining data and the methods that operate on that data into a single unit called a classes. It restricts direct access to an objects internal data and allows interaction

only through defined methods. Encapsulation helps achieve data hiding by preventing unauthorized access and modification of data, thereby improving security, reliability and the integrity of the program.

2. Why are instance variable typically declared as private and how do getters and setters provide controlled access?

Ans:

Instance variable are typically declared as private to protect them from direct access by other classes. This ensures that the internal state of an object cannot be modified arbitrarily, which helps maintain data consistency and security. Getters and Setters provide controlled access to these private variables by allowing programmers to define specific rules for reading and updating data. Through these methods, validation checks, restrictions and additional processing

can be implemented before accessing or modifying the values.

3. How does encapsulation contribute to maintainability and flexibility when modifying internal implementation details of a class?

Ans:

Encapsulation improves maintainability and flexibility by separating the internal implementation of a class from its external interface. Since the internal data and implementation

details are hidden, developers can modify or update the internal logic of a class without affecting other parts of the program that use the class. This reduces dependencies between components, simplifies maintenance and allows changes to be made more efficiently while preserving the overall functionality of the software. Encapsulation also makes debugging and testing easier because changes are confined within the class itself.